Project A Report - Dilanur Bayraktar and Aya Ismail

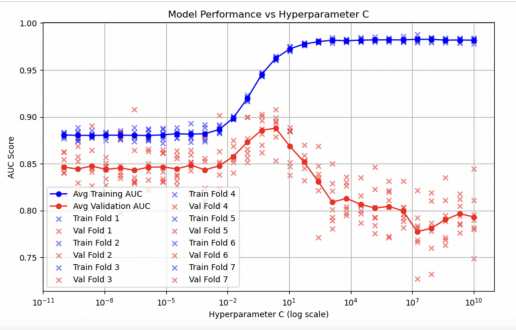
1A: Bag-of-Words Design Decision Description

We cleaned the data using the CountVectorizer's default parameters that converted all uppercase letters to lowercase letters. It also handled punctuation, removing apostrophes. We decided that we wanted to keep numbers as we did not think that removing them made a significant difference to the result. To determine the final vocabulary set, we used the stop-words='english' parameter that excluded common words, such as "a", "the", "at", "in", etc. Here, we noticed that "not" was not in the final vocabulary list. **Because it is a word that provides valuable context to the sentence, we decided to add it to the final vocabulary list, which improved our AUC-ROC score in the long run (see 1D for details).** We also used min-df=6 to exclude all words that appear in less than 6 documents (rare words). By limiting our vocabulary this way, our final vocabulary size we had was 571 words across all folds. When deciding between counts and binary values for the vectorizers, we decided to use counts because we thought that they provided a more accurate representation than whether the word appears in a document or not. We also decided to include the ngram_range=(1, 2) parameter in our vectorizer because through experimentation and looking at the False Negatives and False Positives, we have observed that our model yields more accurate results when it accounts for biagrams (specifically negations, such as "not worth"). Finally, with out-of-vocabulary words in the test set, we decided to ignore them.

1B: Cross Validation Design Description

To best optimize heldout data, we will utilize the area under the ROC curve as our performance metric. For our cross-validation, we manually tested 3, 5, 7, 10 as our number of folds, and decided that 7 yields a better validation accuracy. Therefore, we used StratifiedKFold from sklearn with n_splits=7 for our final model. We then used the cv.split function to split our data into x_tr, x_va, y_tr, and y_va for each of the 7 folds. Additionally, for each fold, we computed predicted values using x_va and x_tr, which were then compared against the y_tr and y_va values to find the training and validation AUC-ROC scores. We stored the training and validation AUC-ROC scores for each fold in an array, which were then used to calculate the average AUC-ROC scores. **To find the best model, we found the maximum average AUC-ROC score across all 7 folds and the c and max_df values that led to it (more details in 1C).**

1C: Hyperparameter Selection for Logistic Regression Classifier



We experimented with different values of C for our model. At first, we had a big range from negative million to positive million, and then we narrowed down the range to logspace(-10, 10, 30), which gave us 30 different C values to experiment with.

Figure 1: AUROC Score vs Hyperparameter C (log scale). The blue curve is the Training AUC and the red curve is the Validation AUC.

As shown in Figure 1, higher values of C result in the AUROC score for the validation set to decrease while the AUROC score for the training set to increase. From the graph, it looks like the best C value that leads to the best AUROC score on the validation set is between 10^{-2} and 10^{1} (log scale). We confirmed this by calculating the AUROC scores across all folds and averaging them for each C value. Then, we found the maximum of the AUROC scores across the C

values and used the same index to find the best C value. **The C value that leads to the greatest AUROC score on the validation set is 0.4520 with a resulting AUC score of 0.8864.**

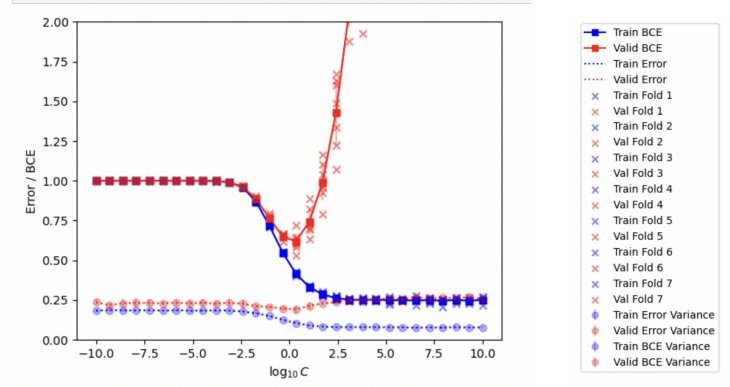


Figure 2: BCE and Error vs Hyperparameter C (log scale). The blue curves are the Training BCE and Error and the red curves are the Validation BCE and Error.

An interesting property that we discovered is that the fluctuation in the validation curve in Figure 1 with the greater values of C is caused by the shuffle = True hyperparamater in the StratifiedKFold. Before we set shuffle to True, we did not observe the fluctuation. We think this is due to the randomness caused by the shuffling. We can observe from the graphs that with lower values of C (-10 to -2.5), the model underfits and the BCE for both the training and validation sets are too high. However, with higher values of C, the model overfits as can be told from the extreme split in the training and validation BCE curves. **Here, the C values that yield the least BCE for both the validation and training set are in the range -1 to 1. The best C value that we obtained from Figure 1 falls within this range, once again confirming our analysis.** Additionally, we tried to explore different hyperparameters, specifically the max_df and min_df in our CountVectorizer to see if the relationship between the C value and max_df/min_df would lead to a more optimized model that would increase the AUROC score. We were able to get the C, max_df, and min_df values that together resulted in a better AUROC score than before. In fact, the values we obtained were C=0.452035, max_df = 0.8, min_df=5. However, once we submitted our test predictions to the Leaderboard, our score decreased. While we were not able to tell why our score decreased, because we had limited time, we decided to bring back our old version of the code and only search through the C hyperparameter.

1D: Analysis of Predictions for the Best Classifier

Below are false positive instances extracted from the validation set for the model with the best hyperparameters (in particular C=0.452035,). **Note**: we only printed the false positives with predicted probabilities greater than 0.6, so treating this value as a threshold.

1. Review: Not nearly as good looking as the AMAZON picture makes it look. True label: 0, **Pred: 0.6970**
2. Review: I highly doubt that anyone could ever like this trash. True label: 0, **Pred: 0.6383**
3. Review: It is **not** good. True label: 0, **Pred: 0.7694**
4. Review: The only place good for this film is in the garbage. True label: 0, **Pred: 0.8487**
5. Review: And, quite honestly, often its **not** very good. True label: 0, **Pred: 0.8338**
6. Review: Soggy and **not** good. True label: 0, **Pred: 0.7694**
7. Review: It was attached to a gas station, and that is rarely a good sign. True label: 0, **Pred: 0.7694**
8. Review: **Not** good by any stretch of the imagination. True label: 0, **Pred: 0.7694**

Associated confusion matrix results for the fold: TP: 155, FP: 16, FN: 63, TN: 108 (used threshold 0.6)

Observations:

- There does not seem to be a clear relationship between the length of the sentence and the prediction from above since the wrong-predicted reviews have a variety of lengths. Furthermore, the wrong predictions are coming from all three datasets so we also think that there is no clear correlation between the type of review (amazon, yelp. imdb) and the prediction.
- Though, we noticed that there are multiple reviews where there is an explicit 'not' where the review is predicted to be positive as shown in examples 7, 10, 13 above. We excluded the word "not" from the stop words and observed the change on the false positive instances as follows: (**Note that AUROC score for validation set has increased to 0.8861 with this change**)
    1. Review: All in all, I'd expected a better consumer experience from Motorola. True label: 0, **Pred: 0.6481**
    2. Review: The internet access was fine, it the rare instance that it worked. True label: 0, **Pred: 0.6843**
    3. Review: Very Dissapointing Performance. True label: 0, **Pred: 0.62**
    4. Review: I highly doubt that anyone could ever like this trash. True label: 0, **Pred: 0.6991**
    5. Review: There was NOTHING believable about it at all. True label: 0, **Pred: 0.6**
    6. Review: When my order arrived, one of the gyros was missing. True label: 0, **Pred: 0.7344**

New Associated Confusion matrix results: TP: 160, FP: 11, FN: 53, TN: 118.

New false positives do not have the same problem as before.

False Negative Instances for the most recent model:

1. Review: Someone shouldve invented this sooner. True label: 1, **Pred: 0.4588**
2. Review: Phone is sturdy as all nokia bar phones are. True label: 1, **Pred: 0.4930**
3. Review: Much less than the jawbone I was going to replace it with. True label: 1, **Pred: 0.2851**
4. Review: So I bought about 10 of these and saved alot of money. True label: 1, **Pred: 0.3927**
5. Review: 2 thumbs up to this seller True label: 1, **Pred: 0.4588**
6. Review: Crisp and Clear. True label: 1, **Pred: 0.5692**

We can generally say that the model we have created is more of a syntactic/explicit rather than a semantic/implicit sentiment analyzer. We can see this for both false positive and negative predictions. We were also able to observe that the model predicted more false negatives than false positives. We understand that the choice of threshold 0.6 caused the confusion matrix to have more false negatives than false positives. However, even if we set the threshold to be 0.5, the model still predicts more false negatives. Our guess for this behavior is because good reviews had less explicit "good" words compared to bad reviews. Examples are "crisp", "you won't forget this movie", "Someone shouldve invented this sooner," and more.

1E : Report Performance on Test Set via Leaderboard

Our final AUROC score on the test set as reported by the Leaderboard is 0.86467. The best score on leaderboard that we had before this score was 0.84. The best validation AUROC score we achieved is 0.886, while it was 0.8675 before this increase. The decrease from the validation AUROC to the test AUROC is reasonable and can be explained by the new words that are in the test data that are not in the bag of words feature representation. The process of cross validation executed made sure that the model does not overfit the training data.

2A: Feature Representation description

BERT Embeddings: **To transform text into fixed-length feature vectors suitable for classification, we used the BERT Embeddings that were provided to us (x_train_BERT_embeddings.npy and x_test_BERT_embeddings.npy) by using np.load.**

We decided to use BERT embeddings because, after experimenting with the BERT notebook that was provided to us, we decided that it's a much better way to represent text as it converts sentences into a series of numerical tokens and then extracts meaningful feature representations from the sentences. Unlike the CountVectorizer implementation, it accounts for the context of the sentence, too.

TD-IDF: We also explored using TD-IDF vectorizer in addition to BERT Embeddings. Once we had a TD-IDF vectorizer, we ran it on our x_tr_list to vectorize it. We tested it on our Logistic Regression model from Part 1, where we replaced the CountVectorizer with TD-IDF vectorizer.

After our analysis, we decided that BERT Embeddings led to higher AUROC scores than TD-IDF. That's why, moving forward, we kept BERT Embeddings as our feature representation for other models.

2B: Cross Validation (or Equivalent) Description

For Part 2, we decided to experiment with different combinations of cross validation designs and models. Here's a summary of our experimentations: Logistic Regression with StratifiedKFold, SVM with StratifiedKFold, MLP with GridSearchCV, KNN with GridSearchCV and StratifiedShuffleSplit.

The first two were the same as what we were doing for Part 1 of the project, where we used StratifiedKFold from sklearn with n_splits=7 for our final model. We then used the cv.split function to split our data into x_tr, x_va, y_tr, and y_va for each of the 7 folds. For the third one, we used GridSearchCV with cv set to 10 and a parameter grid with hidden_layer_sizes, activation, solver, alpha, and learning_rate hyperparameters. However, for the last one, we had cv = StratifiedShuffleSplit(n_splits=10, test_size=0.2, random_state=42), with a split size of 10, test size of 0.2 and random state of 42. And we did GridSearchCV across the number of neighbors, weights, metric, and p (power parameter for the Minkowski metric) parameters for KNN, which we did not have on Part 1. We manually experimented with different n_splits and test_size parameters, but decided that they don't lead to a significant difference in the validation AUC-ROC.

2C: Classifier description with Hyperparameter search

- Logistic Regression with StratifiedKFold: We used the same model and hyperparameter search as we had in Part 1, the only difference was that we were using BERT embeddings instead of CountVectorizer. **Results:**
  - Best AUC-ROC on validation set: 0.96790
  - C value that led to the best AUC-ROC : 0.09236

Figure 3: Model Performance vs Hyperparameter C

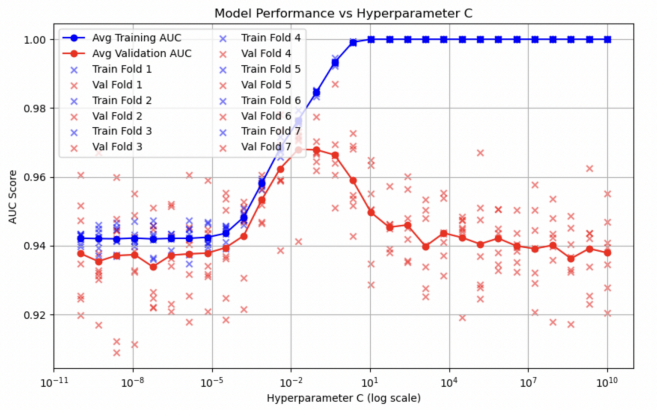
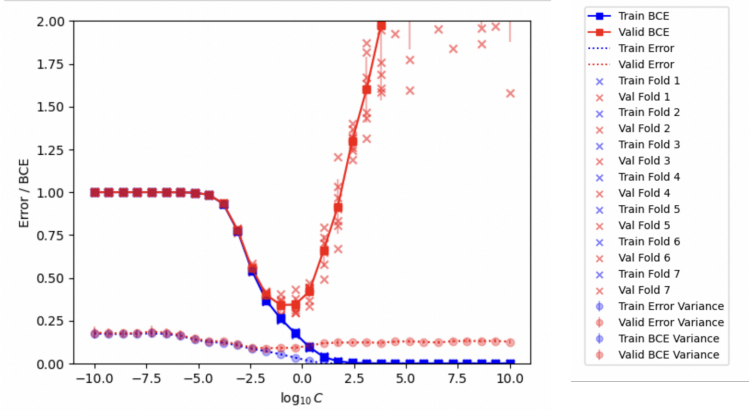


Figure 4: BCE and Error vs Hyperparamater C



As can

be seen in Figures 3 and 4, Logistic Regression with BERT embeddings resulted in a far more accurate model than Part 1. We can also confirm that the C value that led to the best validation AUC-ROC, which is 0.09236, also results in low training and validation BCE, confirming our analysis.

- SVM with StratifiedKFold: We used the same hyperparameter search as we had in Part 1, the only difference was that we were using BERT embeddings with an SVM model, not Logistic Regression. **Results:** Best AUC-ROC on the validation set is 0.967962 and C value that led to the best AUC-ROC is 0.07742. These results again are approximately 10% higher than what we had in Part 1. Surprisingly, the best AUC-ROC on the validation set is close in value to the Logistic Regression with BERT Embeddings score. That's why we decided to submit this model to the Leaderboard to see how it would result in the test set. **We were surprised to see that our score on the Leaderboard decreased with the SVM model.** We think this might have happened because SVM is more prone to overfitting. This is how we determined that LogisticRegression with StratifiedKFold is a better-optimized model than SVM with StratifiedKFold.
- **KNN with GridSearchCV and StratifiedShuffleSplit:** We had two ways of experimenting with KNN for this problem:
    1. Building a pipeline with KNN and StandardScaler, and performing ShuffleSplit (10 splits) and GridSearchCV across the number of neighbors, weights, metric, and p (power parameter for the Minkowski metric) parameters for KNN. We found the best AUC-ROC score with scorer = make_scorer(roc_auc_score, needs_proba=True). The best AUC-ROC we obtained on the validation set was 0.9264, which is lower than what we had for the last two approaches with LogisticRegression and SVM. The hyperparameters that resulted in this best AUC-ROC were the following: Metric - Manhattan, Weight - Distance, Number of neighbors - 21, P - 1.
    2. Using StratifiedShuffleFold with n_splits=10 to manually train the model with a range of number of neighbors (1-29) with the hyperparameters that resulted in the best AUC-ROC for the validation set when we performed GridSearchCV. We then calculated the best AUC-ROC on the validation set across all splits and graphed how the AUC-ROC changed across different numbers of neighbors. **Results:** The highest AUC-ROC on validation set is 0.918 (which is different from 0.9264). We think that this is because we didn't use make_scorer. We computed the AUC-ROC scores for both training and validation sets across each fold and found the best model manually. The best-performing number of neighbors is 29, the same as the last attempt, which is expected.

  We can see that the AUROC score for KNN is lower than Logistic Regression and SVM with BERT Embeddings. We also observed from the plots of the KNN model that it leads to much more overfitting than the rest of the modes. That's why it is not the best optimized model for this problem.

- MLP with GridSearchCV: Similar to KNN, we used GridSearchCV and a parameter grid with hidden_layer_sizes, activation, solver, alpha, and learning_rate hyperparameters. The best model that we obtained from our search had a AUROC score of 0.92065 with 10 splits, which is lower than our best-performing Logistic Regression model with BERT embeddings. As it is not our best-performing model, we did not plot the hyperparameter graphs for it.

Summary: Our best-performing model is Logistic Regression with StratifiedKFold using BERT Embeddings, with a validation AUROC score of 0.96790 (C: 0.09236).

2D: Error analysis:

Confusion Matrix results with a 0.6 threshold: TP: 159, FP: 12, FN: 19, TN: 152

False positive instances:

1. Review: A must study for anyone interested in the "worst sins" of industrial design. True label: 0, **Pred: 0.7475**
2. Review: I've bought $5 wired headphones that sound better than these. True label: 0, **Pred: 0.6114**
3. Review: If you like a loud buzzing to override all your conversations, then this phone is for you! True label: 0, **Pred: 0.6930**
4. Review: And, FINALLY, after all that, we get to an ending that would've been great had it been handled by competent people and not Jerry Falwell. True label: 0, **Pred: 0.8771**
5. Review: Im big fan of RPG games too, but this movie, its a disgrace to any self-respecting RPGer there is. True label: 0, **Pred: 0.7282**
6. Review: It crackles with an unpredictable, youthful energy - but honestly, i found it hard to follow and concentrate on it meanders so badly. True label: 0, **Pred: 0.6864**

We can see the false predictions here because the negative sentiment in the reviews above is very implicit like in reviews 1, 3 and 4. Other reviews had a mixed sentiment but the negative sentiment was generally stronger like in 5 and 6, and we see that the model struggled with such sentences. However, compared to the best model from problem 1 which struggled with reviews that are explicit and should be easily classified as negative, model 2 does not have this issue but rather struggles with sentences that are less straightforward as explained above.

False Negative instances:

1. Review: I ended up sliding it on the edge of my pants or back pockets instead. True label: 1, **Pred: 0.1220**
2. Review: Very good quality though True label: 1, **Pred: 0.4118**
3. Review: Bluetooth range is good - a few days ago I left my phone in the trunk, got a call, and carried the conversation without a hitch. True label: 1, **Pred: 0.3800**
4. Review: You can't beat the price on these. True label: 1, **Pred: 0.1343**
5. Review: I'm still infatuated with this phone. True label: 1, **Pred: 0.3334**
6. Review: Mark my words, this is one of those cult films like Evil Dead 2 or Phantasm that people will still be discovering and falling in love with 20, 30, 40 years down the line. True label: 1, **Pred: 0.2405**

We can make similar observations as we did for false positive predictions.

2E: Report Performance on Test Set via Leaderboard

Our final AUROC score on the test set as reported by Gradescope is 0.96391, which is close to the validation AUROC we obtained from our model. This AUROC score on the test set is definitely more than the test AUROC score we got for Part 1, which was 0.86467. We observed a 10% increase. We believe this is because BERT embeddings account for the context of the sentences and can perform better sentiment analysis than CountVectorizer.